

```
#include<iostream>
```

```
#include<cstdlib>
```

```
#include<ctime>
```

```
using namespace std;
```

```
#define SIZE 10
```

```
#define UP 0
```

```
#define DOWN 1
```

```
#define LEFT 2
```

```
#define RIGHT 3
```

```
class Grid
```

```
{
```

```
public:
```

```
    Grid() {
```

```
        state = 0;
```

```
        dir[UP] = NULL;
```

```
        dir[DOWN] = NULL;
```

```
        dir[LEFT] = NULL;
```

```
        dir[RIGHT] = NULL;
```

```
    }
```

```
    Grid *getDir(int d) { return dir[d]; }
```

```
    int getState() { return state; }
```

```
    void setDir(int d, Grid *g) { dir[d] = g; }
```

```
    void setState(int s) { state = s;}
```

private:

Grid *dir[4]; // Pointers to neighboring grids in four directions

int state; // 0 represents empty space, 1 represents wall

};

struct List

{

public:

List() {

top = 0;

}

/*

Insert an element from top

*/

void addElement(Grid *g) {

if (top < SIZE * SIZE) {

data[top++] = g;

}

}

/*

Remove an element from top and return a pointer pointing to the element.

If the list is empty, return NULL.

*/

Grid *removeElement() {

```

    if (top > 0) {
        return data[--top];
    }
    return NULL;
}

```

```

void printPath() {
    int j;
    for(j = 1; j < top; j++) {
        if(data[j] == data[j - 1]->getDir(UP)) {
            cout << "UP\n";
        } else if(data[j] == data[j - 1]->getDir(DOWN)) {
            cout << "DOWN\n";
        } else if(data[j] == data[j - 1]->getDir(LEFT)) {
            cout << "LEFT\n";
        } else if(data[j] == data[j - 1]->getDir(RIGHT)) {
            cout << "RIGHT\n";
        }
    }
}
}

```

private:

```

    Grid *data[SIZE * SIZE]; // Array of pointers to store the path
    int top;                 // Top index for stack-like behavior
};

```

class Maze

```

{
public:
    Maze() {
        initMaze(SIZE);
    }

    /*
    function initMaze
    Allocate a 2-D linked list with s * s sizes as the map of the maze.
    Inside the maze, every grid with state 0 represents empty space and 1 represents a
    wall.
    The grid in the top-left is the start point and the bottom-right is the finish point.
    Randomly generate 20% walls in the maze.
    Make sure the start point and finish point are 0.
    */
    void initMaze(int s) {
        maze = new Grid[s * s];
        srand(time(0));

        int totalCells = s * s;
        int wallCount = totalCells * 0.2; // 20% walls
        int wallsAdded = 0;

        for (int i = 0; i < s; ++i) {
            for (int j = 0; j < s; ++j) {
                // Initially, all cells are empty (0), and we will assign walls randomly
                maze[i * s + j].setState(0);
            }
        }
    }
};

```

```

// Link to neighboring grids (up, down, left, right)
if (i > 0) maze[i * s + j].setDir(UP, &maze[(i - 1) * s + j]); // Link up
if (i < s - 1) maze[i * s + j].setDir(DOWN, &maze[(i + 1) * s + j]); // Link down
if (j > 0) maze[i * s + j].setDir(LEFT, &maze[i * s + j - 1]); // Link left
if (j < s - 1) maze[i * s + j].setDir(RIGHT, &maze[i * s + j + 1]); // Link right
}
}

```

```

// Ensure the start and finish points are empty

```

```

maze[0].setState(0);
maze[s * s - 1].setState(0);

```

```

// Randomly assign walls to 20% of the cells, excluding start and finish

```

```

while (wallsAdded < wallCount) {

```

```

    int randRow = rand() % s;

```

```

    int randCol = rand() % s;

```

```

// Ensure we don't place walls on the start (0,0) or end (s-1,s-1)

```

```

if ((randRow == 0 && randCol == 0) || (randRow == s - 1 && randCol == s - 1)) {
    continue;
}

```

```

// If the chosen cell is not already a wall, make it a wall

```

```

if (maze[randRow * s + randCol].getState() == 0) {
    maze[randRow * s + randCol].setState(1);
    wallsAdded++;
}

```

```
    }  
    }  
}
```

```
/*
```

```
function getPath
```

This function will find a path between the start point and the finish point.

Return a list containing the path information inside.

If there is no path between two points, the list will be empty.

```
*/
```

```
List *getPath() {
```

```
    List *path = new List();
```

```
    if (findPath(&maze[0], path)) {
```

```
        return path;
```

```
    } else {
```

```
        return new List(); // Return empty list if no path is found
```

```
    }
```

```
}
```

```
bool findPath(Grid *current, List *path) {
```

```
    // If current is null or it's a wall, or already visited (marked with 2), return false
```

```
    if (current == NULL || current->getState() == 1 || current->getState() == 2) {
```

```
        return false;
```

```
    }
```

```
    // Add current to the path
```

```
    path->addElement(current);
```

```

// Mark the current grid as visited (2)
current->setState(2);

// If we reached the bottom-right corner, return true
if (current == &maze[SIZE * SIZE - 1]) {
    return true;
}

// Recursively try to find the path in each direction
if (findPath(current->getDir(RIGHT), path) ||
    findPath(current->getDir(DOWN), path) ||
    findPath(current->getDir(LEFT), path) ||
    findPath(current->getDir(UP), path)) {
    return true;
}

// Backtrack: remove the current from path if no path found
path->removeElement();
return false;
}

void printMaze() {
    for (int i = 0; i < SIZE; ++i) {
        for (int j = 0; j < SIZE; ++j) {
            cout << (maze[i * SIZE + j].getState() == 1 ? "*" : " ");
        }
    }
}

```

```
        cout << endl;
    }
}
```

private:

```
    Grid *maze; // Pointer to a dynamically allocated array of Grid objects
};
```

```
int main() {
    Maze *maze = new Maze();
    maze->printMaze();
    maze->getPath()->printPath();
    return 0;
}
```